Guía básica de Git desde la terminal

Windows y Linux

1. Antes de empezar

Esta guía explica cómo descargar un repositorio de GitHub a tu ordenador, trabajar en él y subir los cambios, todo desde la terminal. Los comandos sirven igual en Windows y en Linux; cuando haya diferencias, se indican.

Git y GitHub no son lo mismo. Git es el programa que guarda el historial de cambios en tu ordenador. GitHub es la web donde se aloja una copia del repositorio para compartirla.

1.1. Instalar Git

Sistema	**Cómo instalarlo**	**Terminal**
Windows	Descargar el instalador de `https://git-scm.com/download/win` y dejar las opciones por defecto. Incluye Git Bash y el Git Credential Manager.	Git Bash (recomendada) o PowerShell
Linux (Ubuntu/Debian)	`sudo apt update && sudo apt install git`	La terminal del sistema

Recomendación para Windows: usad Git Bash (clic derecho en una carpeta → *Open Git Bash here*, o buscarla en el menú Inicio). Así los comandos son exactamente los mismos que en Linux.

1.2. Comprobar que funciona

Abre la terminal y escribe:

```
git --version
```

Si aparece algo como `git version 2.47.0`, ya está instalado. Si dice que el comando no existe, cierra y vuelve a abrir la terminal; si sigue igual, reinstala Git.

2. Moverse por las carpetas desde la terminal

Antes de clonar, hay que situarse con `cd` en la carpeta donde queremos guardar el proyecto. Git creará la carpeta del repositorio dentro de la carpeta en la que estés.

2.1. Comandos básicos

Qué quiero hacer	Linux / Git Bash	PowerShell	CMD
Ver en qué carpeta estoy	`pwd`	`pwd`	`cd`
Ver qué hay en la carpeta	`ls` (`ls -la` para ver ocultos)	`ls` o `dir`	`dir`
Entrar en una carpeta	`cd nombre_carpeta`	`cd nombre_carpeta`	`cd nombre_carpeta`
Subir a la carpeta anterior	`cd ..`	`cd ..`	`cd ..`
Ir a mi carpeta personal	`cd ~`	`cd ~`	`cd %USERPROFILE%`
Crear una carpeta	`mkdir nombre_carpeta`	`mkdir nombre_carpeta`	`mkdir nombre_carpeta`
Limpiar la pantalla	`clear`	`cls` o `clear`	`cls`

Trucos útiles:

- Pulsa Tab para autocompletar nombres de carpetas y archivos.
- Usa las flechas ↑ / ↓ para recuperar comandos anteriores.
- Si un nombre tiene espacios, ponlo entre comillas: `cd "Mis proyectos"`. Mejor aún: no uses espacios ni tildes en carpetas de proyectos.

2.2. Cómo se escriben las rutas

Sistema	Ejemplo de ruta a la carpeta Documentos
Linux	`/home/alumno/Documentos`
Windows en Git Bash	`/c/Users/alumno/Documents`
Windows en PowerShell / CMD	`C:\Users\alumno\Documents`

En Git Bash las barras son `/` y la unidad `C:` se escribe `/c/`.

2.3. Ejemplo: preparar una carpeta de trabajo

Vamos a crear una carpeta `proyectos_git` dentro de Documentos y entrar en ella.

Linux / Git Bash:

```
cd ~/Documents        # en Linux puede ser ~/Documentos
mkdir proyectos_git
cd proyectos_git
pwd                   # comprobamos dónde estamos
```

PowerShell:

```
cd $HOME\Documents
mkdir proyectos_git
cd proyectos_git
pwd
```

A partir de aquí, todo lo que clonemos aparecerá dentro de `proyectos_git`.

3. Configuración inicial y autenticación en GitHub

Para clonar repositorios privados (`git clone`) o subir cambios (`git push`), tu ordenador tiene que identificarse ante GitHub. Hay dos opciones: HTTPS con token o SSH. En esta guía usamos HTTPS con token (PAT), que es la recomendada en redes académicas o corporativas que bloquean el puerto 22 (SSH).

Esta configuración se hace una sola vez por ordenador.

3.1. Generar el token en GitHub

GitHub ya no acepta tu contraseña normal desde la terminal. En su lugar se usa un *Personal Access Token* (PAT), que funciona como una contraseña especial.

1. Entra en GitHub y ve a *Settings* → *Developer settings* → *Personal access tokens* → *Tokens (classic)*.
2. Pulsa *Generate new token (classic)*.
3. Pon un nombre (por ejemplo, `portatil-clase`) y una fecha de caducidad.
4. Marca el permiso *repo*.
5. Pulsa *Generate token* y **copia el token en ese momento**: GitHub no vuelve a mostrarlo.

Importante: el token es como una contraseña: no lo compartas, no lo subas a ningún repositorio ni lo pegues en el código.

3.2. Configurar tu nombre, correo y el gestor de credenciales

Estos datos aparecerán como autor de cada commit. Usa el mismo correo que en tu cuenta de GitHub.

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_email@ejemplo.com"
```

Después, configura dónde se guardará el token para no tener que escribirlo cada vez. Este paso cambia según el sistema:

Sistema	Comando	Qué hace
Windows	`git config -global credential.helper manager`	Usa el Git Credential Manager (instalado con Git) y guarda el token de forma segura en el Administrador de credenciales de Windows.
Linux	`git config -global credential.helper store`	Guarda el token en el archivo `~/.git-credentials`. Es cómodo pero queda en texto plano: úsalo solo en tu ordenador personal.
Linux (equipo compartido)	`git config -global credential.helper "cache -timeout=3600"`	Recuerda el token solo durante 1 hora y no lo guarda en disco.

Para comprobar la configuración:

```
git config --global --list
```

3.3. Qué pasa la primera vez que haces push

La primera vez que Git necesite autenticarse (al hacer `git push` o al clonar un repositorio privado):

- En Windows se abre una ventana del Git Credential Manager. Selecciona la opción *Token* e introduce el PAT que generaste. También puedes elegir *Sign in with your browser* e iniciar sesión en GitHub.
- En Linux la propia terminal pide los datos:
  - `Username`: tu usuario de GitHub.
  - `Password`: **pega el token**, no tu contraseña. Al pegarlo no se ve nada en pantalla; es normal. Pulsa Enter.

A partir de ahí, las credenciales quedan guardadas y no se vuelven a pedir.

3.4. Si el repositorio lo creaste tú en local (sin clonar)

Cuando clonas un repositorio (sección 4), la conexión con GitHub (`origin`) ya viene configurada. Solo si creaste el proyecto en tu ordenador con `git init` tienes que indicar a qué repositorio de GitHub va:

```
# Si el repositorio es nuevo y todavía no tiene remoto:
git remote add origin https://github.com/usuario/repositorio.git

# Si ya tenía un remoto y quieres cambiar la URL (por ejemplo, de SSH a HTTPS):
git remote set-url origin https://github.com/usuario/repositorio.git
```

3.5. Alternativa: SSH

SSH usa un par de claves en lugar de un token. Es muy cómodo en casa, pero suele fallar en la red del centro porque el puerto 22 está bloqueado. Si ves errores como `Connection timed out` al usar una URL del tipo `git@github.com:...`, cambia a HTTPS con `git remote set-url` (sección 3.4).

4. Clonar un repositorio (`git clone`)

Clonar es descargar a tu ordenador una copia completa de un repositorio de GitHub, con todo su historial. Solo se hace una vez por proyecto.

4.1. Copiar la URL del repositorio

1. Abre el repositorio en GitHub.
2. Pulsa el botón verde *<> Code*.
3. Elige la pestaña *HTTPS* y copia la URL. Tiene esta forma: `https://github.com/usuario/repositorio.git`

Para pegar en la terminal: en Git Bash usa Shift + Insert o clic derecho → *Paste*; en PowerShell y en Linux, Ctrl + Shift + V o clic derecho.

4.2. Clonar

Sitúate primero en tu carpeta de trabajo y ejecuta:

```
cd ~/Documents/proyectos_git
git clone https://github.com/usuario/repositorio.git
```

Git crea una carpeta nueva con el nombre del repositorio. Ahora tienes que entrar en ella; si no, los siguientes comandos de Git no funcionarán:

```
cd repositorio
ls          # en CMD: dir
```

4.3. Comprobar que todo está bien

```
git status
git remote -v
```

- `git status` debe decir algo como `On branch main` y `nothing to commit, working tree clean`.
- `git remote -v` muestra la URL de GitHub asociada como `origin`.

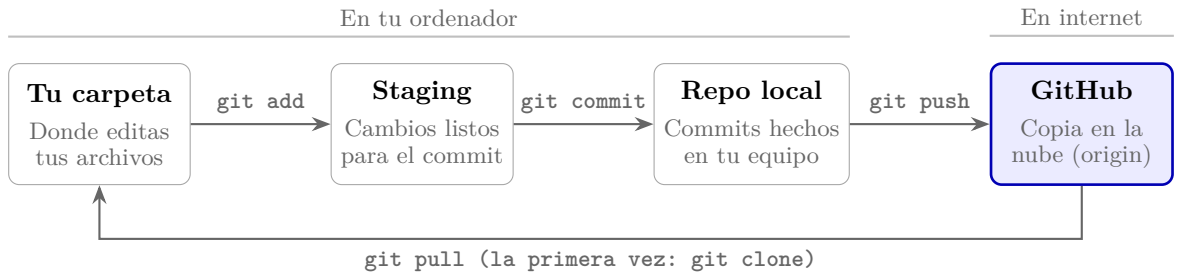
Si aparece `fatal: not a git repository`, no estás dentro de la carpeta del repositorio: usa `pwd` y `cd` para entrar.

5. El ciclo de trabajo: add, commit y push

Cada vez que trabajes en el proyecto repetirás el mismo ciclo: modificar archivos → `git add` → `git commit` → `git push`. Todos estos comandos se ejecutan dentro de la carpeta del repositorio.

5.1. Las cuatro zonas de Git

Un cambio pasa por cuatro sitios antes de llegar a GitHub. Cada comando lo mueve un paso:



Hasta que no haces `git push`, tus cambios solo existen en tu ordenador. Con `git pull` haces el camino inverso: traes a tu carpeta lo que hay en GitHub.

5.2. Paso a paso

1. Ver el estado del repositorio

```
git status
```

Es el comando más importante: úsalo antes y después de cada paso. Muestra en rojo los archivos modificados que aún no has añadido y en verde los que ya están preparados para el commit.

2. Añadir los cambios (staging)

```
git add .                 # añade TODOS los cambios de la carpeta actual
git add index.html        # o solo un archivo concreto
```

El punto `.` significa "todo lo que hay en esta carpeta y sus subcarpetas". Antes de usarlo, revisa con `git status` que no vas a subir nada que no quieras.

3. Guardar los cambios en un commit

```
git commit -m "Añade formulario de contacto"
```

Un commit es una "foto" del proyecto en ese momento. El mensaje entre comillas debe explicar qué has cambiado, en pocas palabras. Mensajes como `cambios` o `asdf` no sirven para nada a quien lea el historial.

Si olvidas el `-m`, Git abre un editor de texto (normalmente Vim). Para salir sin guardar, pulsa `Esc`, escribe `:q!` y pulsa Enter. Vuelve a hacer el commit con `-m`.

4. Subir los commits a GitHub

```
git push
```

Si es un repositorio que creaste tú con `git init` y es la primera vez que subes, usa:

```
git push -u origin main
```

El `-u` deja enlazada tu rama local con la de GitHub, y a partir de ahí basta con `git push`. La primera vez se pedirá el token (sección 3.3).

5. Descargar los cambios de otros (antes de empezar a trabajar)

```
git pull
```

Si trabajas en equipo o desde varios ordenadores, haz `git pull` al empezar cada sesión, para tener la última versión antes de tocar nada.

5.3. Ejemplo completo de una sesión de trabajo

```
cd ~/Documents/proyectos_git/repositorio   # 1. entrar en el proyecto
git pull                                   # 2. traer lo último de GitHub
# ... editas tus archivos en VS Code u otro editor ...
git status                                 # 3. ver qué ha cambiado
git add .                                  # 4. preparar los cambios
git commit -m "Corrige el menú de navegación"  # 5. guardar
git push                                   # 6. subir a GitHub
```

Para comprobarlo, recarga la página del repositorio en GitHub: deberías ver tu commit y tus archivos actualizados.

5.4. Ver el historial

```
git log --oneline
```

Muestra la lista de commits, uno por línea, del más reciente al más antiguo. Si la lista es larga, pulsa `q` para salir.

6. Trabajar con ramas

Este apartado es opcional para la primera práctica. Hasta ahora hemos trabajado siempre en la rama principal, `main`. Una *rama* es una línea de trabajo paralela: permite probar algo nuevo sin estropear lo que ya funciona y, cuando está listo, unirlo a `main`.

6.1. Comandos básicos

Qué quiero hacer	Comando
Ver las ramas y en cuál estoy (marcada con *)	`git branch`
Crear una rama nueva y cambiarme a ella	`git switch -c nombre-rama`
Cambiarme a una rama que ya existe	`git switch nombre-rama`
Subir una rama nueva a GitHub por primera vez	`git push -u origin nombre-rama`
Unir otra rama a la rama en la que estoy	`git merge nombre-rama`
Borrar una rama local ya unida	`git branch -d nombre-rama`

En versiones antiguas de Git, en lugar de `git switch` se usa `git checkout` (`git checkout -b nombre-rama` para crear y cambiar).

6.2. Ejemplo: añadir una funcionalidad en una rama

```
git switch main                  # partimos de main
git pull                         # actualizada
git switch -c formulario-login   # creamos la rama y entramos en ella
# ... editamos archivos ...
git add .
git commit -m "Añade formulario de login"
git push -u origin formulario-login   # la subimos a GitHub

# Cuando está terminada, la unimos a main:
git switch main
git pull
git merge formulario-login
git push
```

En proyectos en equipo, en lugar de hacer `git merge` en local, lo habitual es abrir un *Pull Request* en GitHub para que otra persona revise los cambios antes de unirlos.

Importante: antes de cambiar de rama, haz commit de tus cambios (o Git puede impedirte cambiar). Comprueba siempre con `git status` y `git branch` en qué rama estás antes de hacer commit.

7. Errores frecuentes

Mensaje de error	Qué significa	Cómo solucionarlo
`fatal: not a git repository`	No estás dentro de la carpeta del repositorio.	Usa `pwd` para ver dónde estás y `cd` para entrar en la carpeta del proyecto.
`git: command not found` / *no se reconoce como comando*	Git no está instalado o la terminal se abrió antes de instalarlo.	Cierra y abre la terminal. Si sigue, reinstala Git.
`Authentication failed` / `Invalid username or token`	Has puesto tu contraseña en vez del token, o el token ha caducado.	Genera un token nuevo. En Windows, borra la credencial antigua de *github.com* en el Administrador de credenciales; en Linux, borra la línea de GitHub en `~/.git-credentials`. Vuelve a hacer `git push`.
`Please tell me who you are`	Falta configurar nombre y correo.	Ejecuta los dos `git config -global user...` de la sección de configuración.
`nothing to commit, working tree clean`	No hay cambios nuevos que guardar.	No es un error. Comprueba que has guardado los archivos en el editor.
`rejected ... (fetch first)` al hacer push	En GitHub hay commits que tú no tienes.	Ejecuta `git pull` y después `git push`.
`error: src refspec main does not match any`	No has hecho ningún commit todavía, o tu rama se llama `master`.	Haz un commit primero. Si la rama se llama `master`, renómbrala con `git branch -M main`.
`Connection timed out` en el puerto 22	La red bloquea SSH.	Usa HTTPS con `git remote set-url origin` seguido de la URL HTTPS del repositorio.

8. Chuleta resumen

Una sola vez por ordenador:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_email@ejemplo.com"
git config --global credential.helper manager   # Windows
git config --global credential.helper store     # Linux
```

Una sola vez por proyecto:

```
cd ~/Documents
mkdir proyectos_git
cd proyectos_git
git clone https://github.com/usuario/repositorio.git
cd repositorio
```

Cada vez que trabajes:

```
git pull
# ... trabajas ...
git status
git add .
git commit -m "Mensaje que explica el cambio"
git push
```